SAVITRIBAI PHULE PUNE UNIVERSITY

Dr. D. Y. Patil Technical Campus, Varale, Talegaon, Pune

DR. D. Y. PATIL
VARALE CAMPUS

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

PRACTICAL MANUAL

ADVANCED JAVA PROGRAMMING

PCC-305-ECE

Third Year Electronics and Computer Engineering

Academic Year 2026–2027

SUBJECT TEACHER & HOD

Prof. Prashant Kathole

SUBMITTED BY

Name: _____

Roll No.: _____

Dr. D. Y. Patil Technical Campus

(Savitribai Phule Pune University)

Department of Electronic & Computer Engineering



CERTIFICATE

This is to certify that

_____

(Roll No.: _____)

has successfully completed the necessary practical work for **"Advanced Java Programming"** in a satisfactory manner, under the guidance of **Prof. Prashant Kathole**, as a partial fulfillment of the requirements of Third-Year BE in Electronics & Computer Engineering for the academic year **2026–27**.

_____	_____
Prof. Prashant Kathole	Dr. S. D. Shirbahadurkar
[Subject Incharge & HOD]	[Principal]

Index

PCC-305-ECE: Advanced Java Programming

Sr no	Experiment Name	Date	Teacher Sign
1	To Implement a Java Program Using the Collection Framework, Specifically ArrayList, HashMap, and Iterator, to Store, Search, Update, and Display Student Records.		
2	To Develop a Java Program Using Lambda Expressions and the Stream API to Sort, Filter, and Display Employee Details.		
3	To Develop a Java Application That Establishes Connectivity with a Student Database Using JDBC (Java Database Connectivity) and Performs CRUD (Create, Read, Update, Delete) Operations on Student Records.		
4	To Design and Develop a Web Application Using Java Servlet to Validate the Username and Password Entered by the User and Display an Appropriate Response.		
5	To Develop a JSP Page That Accepts Student Details Through an HTML Form and Displays the Entered Student Information Using JSP Implicit Objects.		
6	To Design and Develop a JSP-Based Web Application That Performs Basic Arithmetic Operations Such as Addition, Subtraction, Multiplication, and Division on Two Numbers Entered by the User.		

EXPERIMENT NO. 1

Student Record Management using the Java Collection Framework

AIM

To implement a Java program using the **Collection Framework**, specifically **ArrayList, HashMap, and Iterator**, to store, search, update, and display student records.

MAPPED OBJECTIVES

- Understand the concept and importance of the Java Collection Framework.
- Use **ArrayList** to store and manage student objects.
- Use **HashMap** to store student records using a unique key.
- Use **Iterator** to traverse and display collection elements.
- Implement operations such as **insert, search, update, and display** on student records.
- Develop a menu-driven Java application using object-oriented programming concepts.

SOFTWARE AND TOOLS USED

Sr. No.	Software/Tool	Purpose
1	Java JDK 8 or above	To compile and execute the Java program
2	IntelliJ IDEA / Eclipse / NetBeans / VS Code	Integrated development environment
3	Command Prompt / Terminal	To compile and run the program
4	Java Collection Framework	To manage student records
5	Operating System	Windows/Linux/macOS

THEORY

1. Java Collection Framework

The **Java Collection Framework (JCF)** provides a set of classes and interfaces for storing and manipulating groups of objects. It provides commonly used data structures such as lists, sets, queues, and maps.

The major advantages of the Collection Framework are:

- Dynamic storage of objects.
- Ready-to-use data structures and algorithms.
- Efficient searching and manipulation.
- Reduced programming effort.
- Improved code reusability.

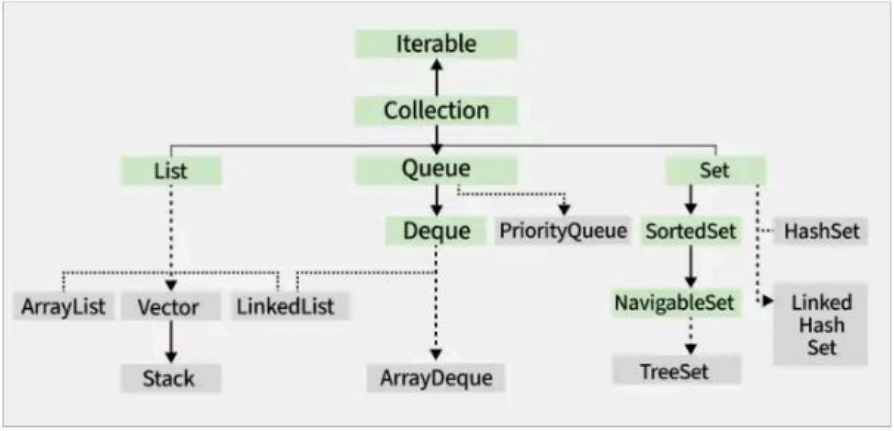


Fig: *Collection Framework Hierarchy*

2. ArrayList

ArrayList is a class that implements the List interface. It stores elements in an ordered sequence and allows duplicate elements.

Syntax:

```
ArrayList<Student> students = new ArrayList<>();
```

Important methods include:

- `add()` – adds an element.
- `get()` – retrieves an element.
- `set()` – updates an element.
- `remove()` – removes an element.
- `size()` – returns the number of elements.

3. HashMap

HashMap stores data in **key-value pairs**. Each key should be unique. In this experiment, the student's roll number can be used as the key.

Syntax:

```
HashMap<Integer, Student> studentMap = new HashMap<>();
```

Important methods include:

- `put()` – inserts or updates a key-value pair.
- `get()` – retrieves a value using its key.
- `containsKey()` – checks whether a key exists.
- `remove()` – removes a record.
- `values()` – returns all values.

4. Iterator

Iterator is used to traverse elements of a collection sequentially.

Syntax:

```
Iterator<Student> iterator = students.iterator();

while (iterator.hasNext()) {
    Student s = iterator.next();
    System.out.println(s);
}
```

Important methods:

- `hasNext()` – checks whether another element exists.
- `next()` – returns the next element.
- `remove()` – removes the current element, when supported.

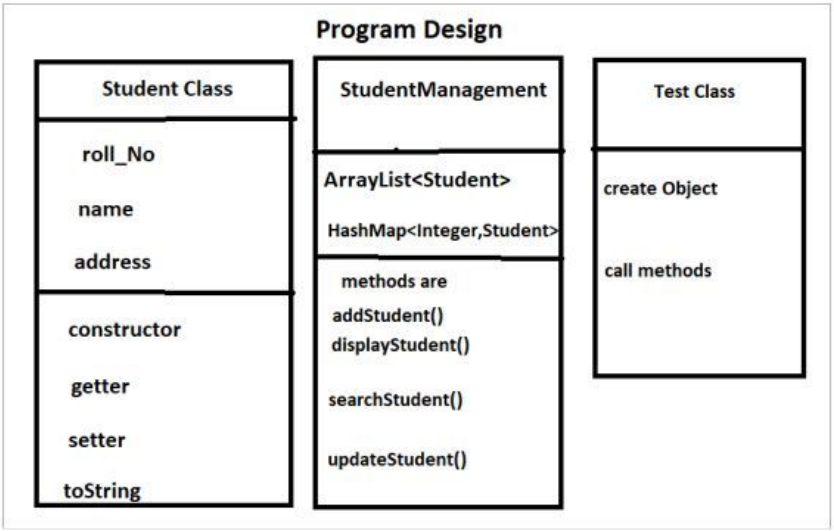


Fig: *Program Design*

CODE

Step 1: Student Class

Student.java JAVA

```
public class Student {

    private int rollNo;
    private String name;
    private String address;

    public Student(int rollNo, String name, String address) {
        this.rollNo = rollNo;
        this.name = name;
        this.address = address;
    }

    public int getRollNo() {
        return rollNo;
    }

    public String getName() {
        return name;
    }
```

```
    public String getAddress() {
        return address;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    @Override
    public String toString() {
        return rollNo + "  " + name + "  " + address;
    }
}
```

Step 2: StudentManagement Class

StudentManagement.java JAVA

```
import java.util.*;

public class StudentManagement {

    ArrayList<Student> studentList = new ArrayList<>();
    HashMap<Integer, Student> studentMap = new HashMap<>();

    // Add Student
    public void addStudent(Student s) {
        // reject a duplicate roll number so that the
        // ArrayList and the HashMap always stay consistent.
        if (studentMap.containsKey(s.getRollNo())) {
            System.out.println("Duplicate Roll No " + s.getRollNo()
                    + " - record not added");
            return;
        }
        studentList.add(s);
        studentMap.put(s.getRollNo(), s);
    }

    // Display using Iterator
    public void displayStudents() {
        Iterator<Student> itr = studentList.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
    }

    // Search Student
    public void searchStudent(int rollNo) {
        Student s = studentMap.get(rollNo);
        if (s != null)
            System.out.println(s);
        else
            System.out.println("Student Not Found");
    }

    // Update Student
```

```
    public void updateStudent(int rollNo,
                              String newName,
                              String newAddress) {
        Student s = studentMap.get(rollNo);
        if (s != null) {
            s.setName(newName);
            s.setAddress(newAddress);
            System.out.println("Record Updated Successfully");
        }
        else {
            System.out.println("Student Not Found");
        }
    }
}
```

Step 3: Test Class

Test.java JAVA

```
public class Test {

    public static void main(String[] args) {

        StudentManagement sm = new StudentManagement();

        sm.addStudent(new Student(101,
                "Amit",
                "Pune"));

        sm.addStudent(new Student(102,
                "Rahul",
                "Mumbai"));

        sm.addStudent(new Student(103,
                "Neha",
                "Nagpur"));

        System.out.println("-----Student Records-----");
        sm.displayStudents();

        System.out.println("\nSearch Roll No 102");
        sm.searchStudent(102);

        System.out.println("\nUpdate Student");
        sm.updateStudent(102,
                "Rahul Sharma",
                "Nashik");

        System.out.println("\nUpdated Records");
        sm.displayStudents();
    }
}
```

ALGORITHM

1. Start.
2. Create the `Student` class with `rollNo`, `name` and `address`, with a constructor, getters, setters and `toString()`.

3. Create the `StudentManagement` class holding an `ArrayList<Student>` and a `HashMap<Integer, Student>` (key = roll number).
4. **Add:** if the roll number already exists in the map, display a duplicate message; otherwise add the student to the list and put it in the map.
5. **Display:** obtain an `Iterator` of the list and print every student while `hasNext()` is true.
6. **Search:** get the student from the map using the roll number; print the record or "Student Not Found".
7. **Update:** get the student from the map; if found, set the new name and address and print a success message, otherwise print "Student Not Found".
8. In `main()` of `Test`: add three students, display all, search roll number 102, update it, and display all again.
9. Stop.

COMPILATION AND EXECUTION

Save the three files in one folder, open a command prompt in that folder and compile all classes with `javac`. Compilation succeeds without errors and creates `Student.class`, `StudentManagement.class` and `Test.class`. Run the program with `java Test`.

Command Prompt COMPILE AND RUN

```
$ javac Student.java StudentManagement.java Test.java
$ java Test
-----Student Records-----
101  Amit  Pune
102  Rahul  Mumbai
103  Neha  Nagpur

Search Roll No 102
102  Rahul  Mumbai

Update Student
Record Updated Successfully

Updated Records
101  Amit  Pune
102  Rahul Sharma  Nashik
103  Neha  Nagpur
```

OUTPUT



Output

Test CONSOLE OUTPUT

```
-----Student Records-----
101  Amit  Pune
102  Rahul  Mumbai
103  Neha  Nagpur

Search Roll No 102
102  Rahul  Mumbai

Update Student
Record Updated Successfully

Updated Records
101  Amit  Pune
102  Rahul Sharma  Nashik
103  Neha  Nagpur
```

CONCLUSION

VIVA QUESTIONS

Basic Questions

1. What is the Java Collection Framework?
2. What is the difference between a collection and a collection framework?
3. What is an ArrayList?
4. What are the advantages of using ArrayList?
5. Can ArrayList contain duplicate elements?
6. What is a HashMap?
7. How does HashMap store data?
8. Why is a unique roll number suitable as a HashMap key?
9. Can a HashMap contain duplicate keys?
10. What happens when put() is called with an existing key?

EXPERIMENT NO. 2

Employee Management using Java 8 Lambda Expressions and Stream API

AIM

To develop a Java program using **Lambda Expressions** and the **Stream API** to sort, filter, and display employee details.

MAPPED OBJECTIVES

- Understand the important features introduced in **Java 8**.
- Understand and implement **Lambda Expressions**.
- Use the **Stream API** to process collections of employee objects.
- Apply filtering operations based on employee attributes.
- Sort employee records using Lambda Expressions and Stream API.
- Display processed employee details efficiently.

SOFTWARE AND TOOLS USED

Sr. No.	Software/Tool	Purpose
1	Java JDK 8 or above	To compile and execute the Java program
2	IntelliJ IDEA / Eclipse / NetBeans / VS Code	To write and execute Java programs
3	Command Prompt / Terminal	To compile and run the program
4	Java Collection Framework	To store employee records
5	Operating System	Windows/Linux/macOS

THEORY

1. Java 8 Features

Java 8 introduced several important features that made Java programming more concise and powerful. Some major features include:

- Lambda Expressions
- Stream API
- Functional Interfaces
- Default and Static methods in interfaces

In this experiment, **Lambda Expressions** and **Stream API** are used for employee record processing.

2. Lambda Expression

A **Lambda Expression** is an anonymous function that can be used to represent the implementation of a functional interface.

General syntax:

```
(parameters) -> expression

or

(parameters) -> {
    statements;
}
```

Example:

```
employees.forEach(emp -> System.out.println(emp));
```

Here, `emp -> System.out.println(emp)` is a Lambda Expression.

Advantages of Lambda Expressions

- Reduces the amount of code.
- Improves code readability.
- Supports functional programming.
- Can be used with collections and Stream API.
- Makes sorting and filtering operations easier to implement.

3. Stream API

The **Stream API** was introduced in Java 8 to process collections of objects in a declarative and efficient manner. A stream does not store data. Instead, it processes data obtained from a collection or another source.

Common Stream operations include:

- `filter()` – selects elements according to a condition.
- `sorted()` – sorts elements.
- `map()` – transforms elements.
- `forEach()` – performs an action on each element.
- `collect()` – collects processed elements into a collection.
- `count()` – counts the number of elements.

Example:

```
employees.stream()
         .filter(emp -> emp.getSalary() > 50000)
         .forEach(emp -> System.out.println(emp));
```

4. Filter Operation

The `filter()` method is used to select elements that satisfy a specified condition.

Example:

```
employees.stream()
         .filter(emp -> emp.getSalary() > 50000)
         .forEach(System.out::println);
```

The above code displays employees whose salary is greater than 50,000.

5. Sorting Using Stream API

The `sorted()` method can be used with a Lambda Expression to sort employee records.

Example:

```
employees.stream()
         .sorted((e1, e2) -> Double.compare(e1.getSalary(), e2.getSalary()))
         .forEach(System.out::println);
```

This sorts employees in ascending order of salary.

For descending order:

```
employees.stream()
         .sorted((e1, e2) -> Double.compare(e2.getSalary(), e1.getSalary()))
         .forEach(System.out::println);
```

PROBLEM STATEMENT

Develop a menu-driven Java application for managing employee records. The application should maintain employee details such as:

- Employee ID
- Employee Name
- Department
- Salary

The program should use **Lambda Expressions** and **Stream API** to:

- Display all employees.
- Filter employees based on salary.
- Filter employees based on department.
- Sort employees by salary.
- Sort employees by name.
- Display the filtered and sorted records.

CODE

EmployeeManagement.java JAVA

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class Employee {

    private int id;
    private String name;
    private String department;
    private double salary;

    // Constructor
    public Employee(int id, String name, String department, double salary) {
        this.id = id;
```

```
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    // Getter methods
    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getDepartment() {
        return department;
    }

    public double getSalary() {
        return salary;
    }

    // toString() method
    @Override
    public String toString() {
        return "Employee ID: " + id +
               ", Name: " + name +
               ", Department: " + department +
               ", Salary: " + salary;
    }
}

public class EmployeeManagement {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Creating employee list
        List<Employee> employees = new ArrayList<>();

        // Adding employee records
        employees.add(new Employee(101, "Rahul", "IT", 65000));
        employees.add(new Employee(102, "Priya", "HR", 48000));
        employees.add(new Employee(103, "Amit", "IT", 75000));
        employees.add(new Employee(104, "Sneha", "Finance", 55000));
        employees.add(new Employee(105, "Karan", "HR", 42000));

        int choice;

        do {
            System.out.println("\n========== EMPLOYEE MANAGEMENT ==========");
            System.out.println("1. Display All Employees");
            System.out.println("2. Filter Employees by Salary");
            System.out.println("3. Filter Employees by Department");
            System.out.println("4. Sort Employees by Salary");
            System.out.println("5. Sort Employees by Name");
            System.out.println("6. Exit");
            System.out.print("Enter your choice: ");

            choice = sc.nextInt();
```

```
            sc.nextLine();

            switch (choice) {

                case 1:
                    System.out.println("\n----- ALL EMPLOYEES -----");
                    employees.stream()
                             .forEach(System.out::println);
                    break;

                case 2:
                    System.out.print("Enter minimum salary: ");
                    double minimumSalary = sc.nextDouble();

                    System.out.println("\n----- FILTERED EMPLOYEES -----");
                    employees.stream()
                             .filter(emp -> emp.getSalary() >= minimumSalary)
                             .forEach(System.out::println);
                    break;

                case 3:
                    System.out.print("Enter department: ");
                    String department = sc.nextLine();

                    System.out.println("\n----- EMPLOYEES IN " +
                                       department + " DEPARTMENT -----");
                    employees.stream()
                             .filter(emp ->
                                     emp.getDepartment()
                                        .equalsIgnoreCase(department))
                             .forEach(System.out::println);
                    break;

                case 4:
                    System.out.println("\n----- EMPLOYEES SORTED BY SALARY -----");
                    employees.stream()
                             .sorted((e1, e2) ->
                                     Double.compare(
                                         e1.getSalary(),
                                         e2.getSalary()))
                             .forEach(System.out::println);
                    break;

                case 5:
                    System.out.println("\n----- EMPLOYEES SORTED BY NAME -----");
                    employees.stream()
                             .sorted((e1, e2) ->
                                     e1.getName()
                                       .compareToIgnoreCase(e2.getName()))
                             .forEach(System.out::println);
                    break;

                case 6:
                    System.out.println("Exiting Employee Management System...");
                    break;

                default:
                    System.out.println("Invalid choice. Please try again.");
            }

        } while (choice != 6);
```

```
        sc.close();
    }
}
```

COMPILE AND RUN

```
javac EmployeeManagement.java
java EmployeeManagement
```

EXPLANATION OF IMPORTANT CODE STATEMENTS

1. Creating Employee List

```
List<Employee> employees = new ArrayList<>();
```

An ArrayList is used to store multiple employee objects.

2. Filtering Employees

```
employees.stream()
         .filter(emp -> emp.getSalary() >= minimumSalary)
         .forEach(System.out::println);
```

The `filter()` operation selects employees whose salary is greater than or equal to the specified salary.

3. Filtering by Department

```
employees.stream()
         .filter(emp ->
                 emp.getDepartment()
                    .equalsIgnoreCase(department))
         .forEach(System.out::println);
```

This displays employees belonging to the specified department.

4. Sorting by Salary

```
employees.stream()
         .sorted((e1, e2) ->
                 Double.compare(e1.getSalary(), e2.getSalary()))
         .forEach(System.out::println);
```

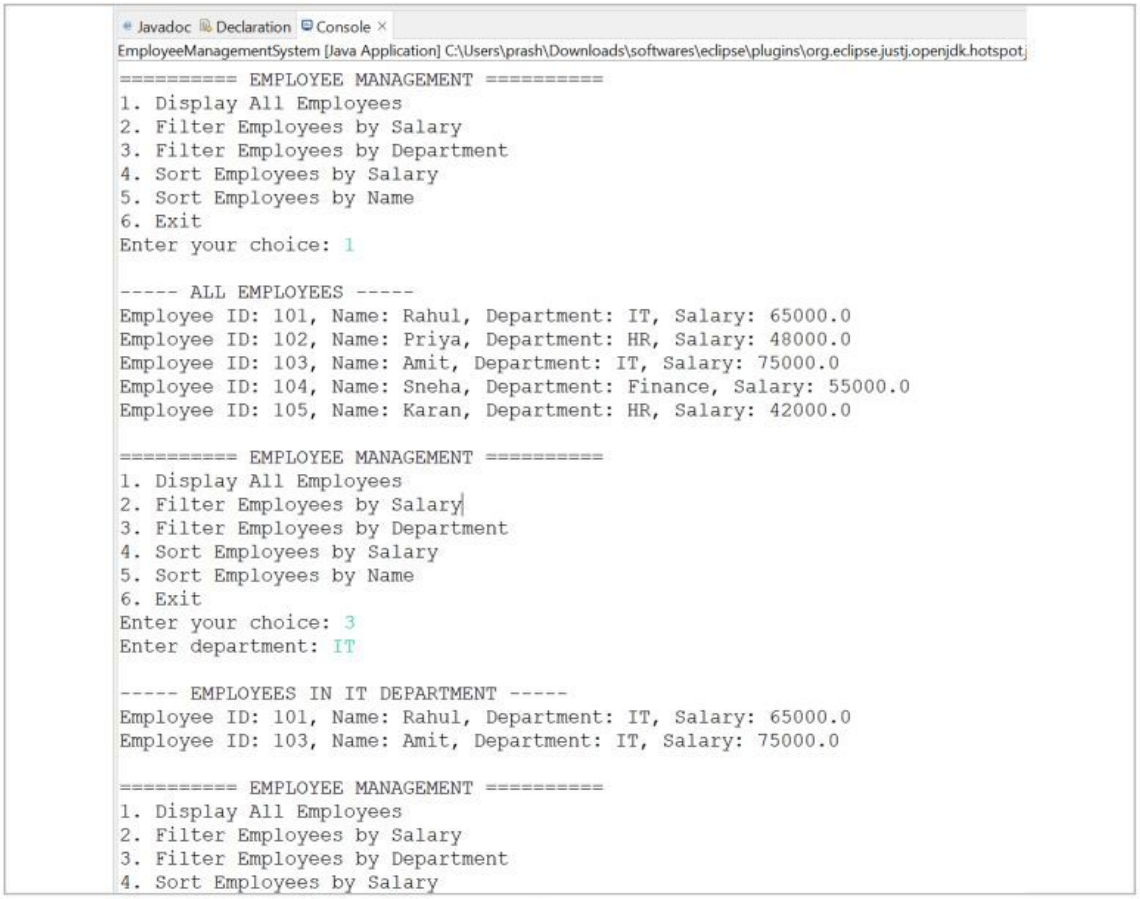
This sorts employees in ascending order of salary.

5. Sorting by Name

```
employees.stream()
         .sorted((e1, e2) ->
                 e1.getName().compareToIgnoreCase(e2.getName()))
         .forEach(System.out::println);
```

This sorts employee records alphabetically according to employee name.

OUTPUT



Output

EmployeeManagement CONSOLE OUTPUT

```
========== EMPLOYEE MANAGEMENT ==========
1. Display All Employees
2. Filter Employees by Salary
3. Filter Employees by Department
4. Sort Employees by Salary
5. Sort Employees by Name
6. Exit
Enter your choice: 1
----- ALL EMPLOYEES -----
Employee ID: 101, Name: Rahul, Department: IT, Salary: 65000.0
Employee ID: 102, Name: Priya, Department: HR, Salary: 48000.0
Employee ID: 103, Name: Amit, Department: IT, Salary: 75000.0
Employee ID: 104, Name: Sneha, Department: Finance, Salary: 55000.0
Employee ID: 105, Name: Karan, Department: HR, Salary: 42000.0

[menu displayed again - omitted]
Enter your choice: 2
Enter minimum salary: 55000
----- FILTERED EMPLOYEES -----
Employee ID: 101, Name: Rahul, Department: IT, Salary: 65000.0
Employee ID: 103, Name: Amit, Department: IT, Salary: 75000.0
Employee ID: 104, Name: Sneha, Department: Finance, Salary: 55000.0

[menu displayed again - omitted]
Enter your choice: 3
Enter department: it
```

```
----- EMPLOYEES IN it DEPARTMENT -----
Employee ID: 101, Name: Rahul, Department: IT, Salary: 65000.0
Employee ID: 103, Name: Amit, Department: IT, Salary: 75000.0

[menu displayed again - omitted]
Enter your choice: 4
----- EMPLOYEES SORTED BY SALARY -----
Employee ID: 105, Name: Karan, Department: HR, Salary: 42000.0
Employee ID: 102, Name: Priya, Department: HR, Salary: 48000.0
Employee ID: 104, Name: Sneha, Department: Finance, Salary: 55000.0
Employee ID: 101, Name: Rahul, Department: IT, Salary: 65000.0
Employee ID: 103, Name: Amit, Department: IT, Salary: 75000.0

[menu displayed again - omitted]
Enter your choice: 5
----- EMPLOYEES SORTED BY NAME -----
Employee ID: 103, Name: Amit, Department: IT, Salary: 75000.0
Employee ID: 105, Name: Karan, Department: HR, Salary: 42000.0
Employee ID: 102, Name: Priya, Department: HR, Salary: 48000.0
Employee ID: 101, Name: Rahul, Department: IT, Salary: 65000.0
Employee ID: 104, Name: Sneha, Department: Finance, Salary: 55000.0

[menu displayed again - omitted]
Enter your choice: 7
Invalid choice. Please try again.

[menu displayed again - omitted]
Enter your choice: 6
Exiting Employee Management System...
```

CONCLUSION

VIVA QUESTIONS

Basic Questions

1. What are the major features introduced in Java 8?
2. What is a Lambda Expression?
3. What is the syntax of a Lambda Expression?
4. What is a functional interface?
5. Give an example of a functional interface in Java.
6. What is the Stream API?
7. Why was the Stream API introduced in Java?
8. What is the difference between a Collection and a Stream?

9. Does a Stream store data?
10. What are intermediate and terminal operations in a Stream?

Lambda Expression Questions

1. What is the purpose of Lambda Expressions?
2. What are the advantages of Lambda Expressions?
3. How does a Lambda Expression reduce code complexity?
4. Can a Lambda Expression be used without a functional interface?
5. What is the difference between a Lambda Expression and a normal method?
6. Explain the following statement: `emp -> emp.getSalary() > 50000`

Stream API Questions

1. What is the filter() method?
2. What is the sorted() method?
3. What is the forEach() method?
4. What is the purpose of collect()?
5. What is the difference between filter() and map()?
6. What is a terminal operation?
7. Can a Stream be reused after a terminal operation?
8. What happens if no element satisfies the filter() condition?

EXPERIMENT NO. 3

JDBC Connectivity and CRUD Operations

AIM

To develop a Java application that establishes connectivity with a student database using JDBC (Java Database Connectivity) and performs CRUD (Create, Read, Update, Delete) operations on student records.

MAPPED OBJECTIVES

After completing this practical, students will be able to:

- Understand the concept and architecture of JDBC for database connectivity in Java.
- Establish a connection between a Java application and a relational database using JDBC.
- Implement CRUD operations to insert, retrieve, update, and delete student records.
- Execute SQL queries using JDBC interfaces such as Connection, Statement, PreparedStatement, and ResultSet.
- Apply exception handling and proper resource management during database operations.

SOFTWARE REQUIREMENTS

- JDK 8 or above
- MySQL Server (with the `mysql` command-line client or MySQL Workbench)
- MySQL Connector/J (JDBC driver JAR)
- Eclipse / IntelliJ IDEA / NetBeans or a command prompt
- Windows / Linux / macOS

THEORY

3.1 Introduction to JDBC

JDBC (Java Database Connectivity) is a Java API that enables Java applications to interact with relational databases such as MySQL, Oracle, and PostgreSQL.

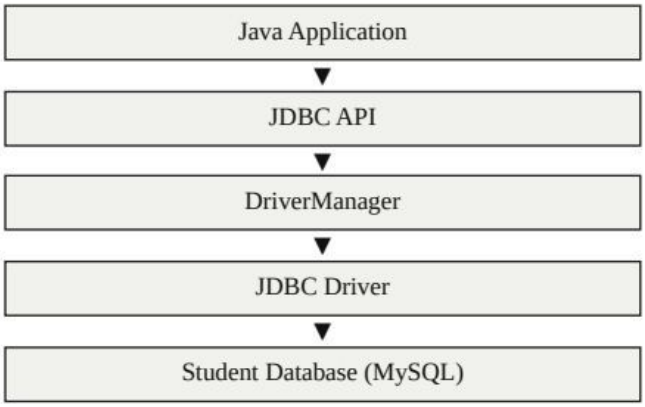
JDBC provides a standard set of classes and interfaces to establish database connections, execute SQL queries, retrieve results, and manipulate database records.

The JDBC API is mainly available through the `java.sql` package.

3.2 JDBC Architecture

JDBC follows a layered architecture consisting of the following components:

- **Java Application:** The application that performs database operations.
- **JDBC API:** Provides interfaces and classes for database connectivity.
- **JDBC DriverManager:** Manages JDBC drivers and establishes connections.
- **JDBC Driver:** Converts JDBC calls into database-specific operations.
- **Database:** Stores and manages student information.



3.3 Important JDBC Interfaces and Classes

Interface / Class	Description
`DriverManager`	Establishes a connection with the database.
`Connection`	Represents an active connection between Java and the database.
`Statement`	Executes static SQL queries.
`PreparedStatement`	Executes parameterized SQL queries efficiently and securely.
`ResultSet`	Stores and processes records retrieved from the database.
`SQLException`	Handles database-related errors and exceptions.

3.4 CRUD Operations Using JDBC

CRUD represents four fundamental database operations:

1. **Create (Insert):** Used to insert new student records into the database. Student details such as roll number, name, department, and marks can be stored using an SQL INSERT statement.
2. **Read (Select):** Used to retrieve student records from the database using the SQL SELECT statement. The retrieved records are processed using the ResultSet interface.
3. **Update:** Used to modify existing student information such as name, department, or marks using the SQL UPDATE statement.
4. **Delete:** Used to remove an existing student record from the database using the SQL DELETE statement.

3.5 Steps to Establish JDBC Connectivity

The following steps are generally followed while developing a JDBC application:

- Import the required JDBC packages.
- Load or register the JDBC driver, if required.
- Establish a database connection using DriverManager.
- Create a Statement or PreparedStatement object.
- Execute SQL queries to perform CRUD operations.
- Process the results using ResultSet, if applicable.
- Close the database resources after completing the operations.

3.6 Important JDBC Methods

Method	Purpose
`getConnection()`	Establishes a database connection.
`prepareStatement()`	Creates a parameterized SQL statement.
`executeQuery()`	Executes a SELECT query and returns a ResultSet.
`executeUpdate()`	Executes INSERT, UPDATE, or DELETE operations.
`next()`	Moves the ResultSet cursor to the next record.
`close()`	Closes JDBC resources.

3.7 Advantages of JDBC

- Provides platform-independent database connectivity.
- Supports multiple relational database systems.
- Enables efficient execution of SQL queries.
- Supports secure database operations using PreparedStatement.
- Provides exception handling and transaction management.

DATABASE SETUP

The SQL below creates the exact database, table and sample rows that `EmployeeJDBC.java` expects. Run it once in the MySQL client (for example `mysql -u your_mysql_user -p < schema.sql`). The three sample rows are taken from the records visible in the supplied output; the salary of record 103 is set to 65000 so that the update step is visible.

schema.sql SQL

```sql
-- Database and table expected by EmployeeJDBC.java
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE IF NOT EXISTS employee7 (
    eid     INT          PRIMARY KEY,
    ename   VARCHAR(50)  NOT NULL,
    salary  DOUBLE       NOT NULL,
    address VARCHAR(100)
);

-- Sample rows (record 103 must exist for the UPDATE step)
INSERT INTO employee7 (eid, ename, salary, address) VALUES
    (103, 'Sneha',     65000, 'Nashik'),
    (107, 'Snehalata',  6500, 'Nagpur'),
    (110, 'ayesha',    80000, 'pune');
```

DRIVER AND CONNECTION SETUP

- Download **MySQL Connector/J** from the MySQL website and add the JAR to the project: in Eclipse use *Project → Properties → Java Build Path → Libraries → Add External JARs*; on the command line add it to the class path with `-cp`.

- Do not write real credentials in the source. Set the environment variables `DB_USER` and `DB_PASSWORD` (and `DB_URL` if the server is not on localhost:3306) before running; otherwise the placeholders in the code are used and the connection is refused.
- With MySQL 8 and `useSSL=false`, the server may ask for `allowPublicKeyRetrieval=true` in the URL if the account uses the default authentication plugin.

CODE

Source file: `Unit2JDBC/EmployeeJDBC.java` (the `package Unit2JDBC;` line requires the file to sit in a folder named `Unit2JDBC`).

Unit2JDBC/EmployeeJDBC.java JAVA

```java
package Unit2JDBC;

import java.sql.*;

public class EmployeeJDBC {

    // connection details are read from environment variables
    // (placeholders are used when a variable is not set), so no real
    // credentials are written in the source.
    static final String URL = System.getenv().getOrDefault("DB_URL",
            "jdbc:mysql://localhost:3306/college?useSSL=false");
    static final String USER = System.getenv().getOrDefault("DB_USER",
            "your_mysql_user");
    static final String PASSWORD = System.getenv().getOrDefault("DB_PASSWORD",
            "your_mysql_password");

    public static void main(String[] args) {

        // try-with-resources closes the Connection even
        // when an exception occurs (the original closed it by hand).
        try (Connection con =
                     DriverManager.getConnection(URL, USER, PASSWORD)) {

            System.out.println("Database connected successfully");

            // CREATE
            String insertSQL =
                    "INSERT INTO employee7 " +
                    "(eid, ename, salary, address) " +
                    "VALUES (?, ?, ?, ?)";

            try (PreparedStatement insert = con.prepareStatement(insertSQL)) {
                insert.setInt(1, 111);
                insert.setString(2, "alia");
                insert.setDouble(3, 60000);
                insert.setString(4, "Mumbai");

                int inserted = insert.executeUpdate();
                System.out.println(inserted + " record inserted");
            }

            // READ
            String selectSQL = "SELECT * FROM employee7";

            try (PreparedStatement select = con.prepareStatement(selectSQL);
                 ResultSet rs = select.executeQuery()) {
```

```
                System.out.println("\nEmployee Records:");

                while (rs.next()) {
                    System.out.println(
                            rs.getInt("eid") + " " +
                            rs.getString("ename") + " " +
                            rs.getDouble("salary") + " " +
                            rs.getString("address"));
                }
            }

            // UPDATE
            String updateSQL =
                    "UPDATE employee7 " +
                    "SET salary = ? " +
                    "WHERE eid = ?";

            try (PreparedStatement update = con.prepareStatement(updateSQL)) {
                update.setDouble(1, 70000);
                update.setInt(2, 103);

                int updated = update.executeUpdate();
                System.out.println("\n" + updated + " record updated");
            }

            // DELETE
            String deleteSQL =
                    "DELETE FROM employee7 " +
                    "WHERE eid = ?";

            try (PreparedStatement delete = con.prepareStatement(deleteSQL)) {
                delete.setInt(1, 111);

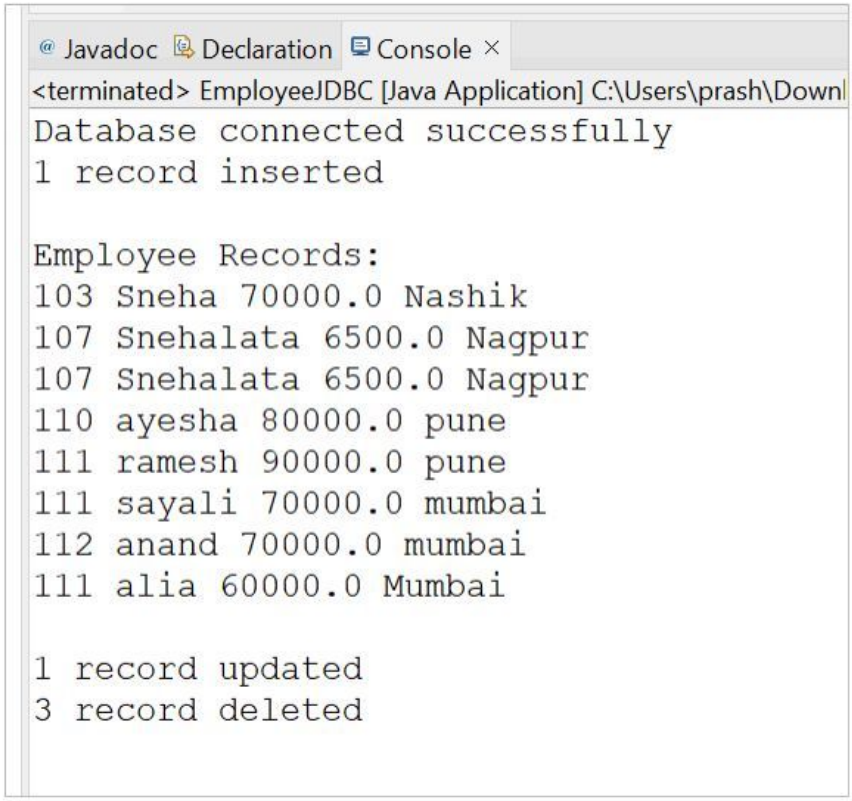
                int deleted = delete.executeUpdate();
                System.out.println(deleted + " record deleted");
            }
        }
        catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```
javac -d out Unit2JDBC/EmployeeJDBC.java
java -cp out:mysql-connector-j-<version>.jar Unit2JDBC.EmployeeJDBC
# On Windows use ";" instead of ":" in the class path.
```

TEST DATA AND EXPECTED RESULTS

Step	SQL operation	Data	Result with `schema.sql`
Create	INSERT	eid 111, alia, 60000, Mumbai	1 record inserted
Read	SELECT *	—	Lists 103, 107, 110 and the new record 111
Update	UPDATE … WHERE eid = ?	salary 70000 for eid 103	1 record updated
Delete	DELETE … WHERE eid = ?	eid 111	1 record deleted

OUTPUT



Output

EmployeeJDBC CONSOLE OUTPUT

```
Database connected successfully
1 record inserted

Employee Records:
103 Sneha 65000.0 Nashik
107 Snehalata 6500.0 Nagpur
110 ayesha 80000.0 pune
111 alia 60000.0 Mumbai

1 record updated
1 record deleted
```

CONCLUSION

VIVA QUESTIONS

1. What is JDBC and which package provides it?
2. What are the steps to connect a Java program to a database?
3. What is the difference between `Statement` and `PreparedStatement`?
4. What is the difference between `executeQuery()` and `executeUpdate()`?
5. What does a `ResultSet` cursor do and what does `next()` return?
6. Why should JDBC resources be closed, and how does try-with-resources help?

EXPERIMENT NO. 4

Login Validation using Servlet

AIM

To design and develop a web application using **Java Servlet** to validate the username and password entered by the user and display an appropriate response.

PROBLEM STATEMENT

Design a web application that accepts **username** and **password** from the user. The Servlet should validate the entered credentials. If the username and password are correct, display a **Welcome** message; otherwise, display an **Invalid Username or Password** message.

SOFTWARE REQUIREMENTS

- JDK 11 or above
- Apache Tomcat Server **10.1.x**
- Eclipse IDE / NetBeans / IntelliJ IDEA
- HTML5
- Java Servlet
- Web Browser
- MySQL Server and MySQL Connector/J

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Internet connection is not mandatory

THEORY

A Servlet is a technology that is very much beneficial for creating dynamic web pages. It acts as an intermediate layer between the web browser and HTTP clients or servers. With the help of servlets, at first, we collect the inputs from the users and then create web pages dynamically. Servlet extends the capabilities of servers and host applications. Servlets extend the capabilities of the servers and also respond to any type of requests for users.

1. The Servlet Life Cycle

The servlet life cycle is managed entirely by the **Servlet Container** (like Apache Tomcat). It consists of four main phases:

- **Loading and Instantiation:** The container finds the servlet class, loads it into memory, and creates an instance of it using its default constructor.
- **Initialization (`init()` method):** The container calls the `init()` method exactly **once** after instantiation. It is used for one-time initialization tasks (e.g., opening a database connection).

- **Request Handling (`service()` method):** For every client request, the container spawns a new thread and calls the `service()` method. This method determines the type of HTTP request (GET, POST, PUT, DELETE) and dispatches it to the appropriate handler method like `doGet()` or `doPost()`.
- **Destruction (`destroy()` method):** The container calls the `destroy()` method exactly **once** before removing the servlet instance from service. It is used to clean up resources (e.g., closing database connections).

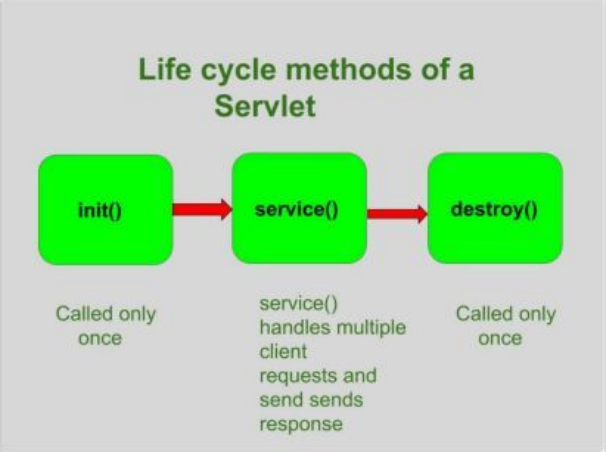


Fig: *Life cycle methods of a Servlet*

2. Key Objects & Session Management

When a request comes in, the container creates two vital objects and passes them to the servlet:

- **HttpServletRequest:** Contains all incoming client data (parameters, headers, cookies, URL).
- **HttpServletResponse:** Used to configure and send the response back to the client (content type, HTML body, status codes).

To maintain state between multiple requests from the same user, servlets use **Session Tracking Mechanisms**:

- **HttpSession:** An API used to store user-specific data across multiple page requests.
- **Cookies:** Small text files stored on the client side that can pass a unique Session ID (JSESSIONID) back to the server.
- **URL Rewriting:** Appending the Session ID to the end of URLs when cookies are disabled.

3. Requests Mapping

For a web server to know which servlet should handle a specific URL, you must map the URL to the servlet. This can be done in two ways:

- **Annotation-based (Modern):** Using the `@WebServlet("/myurl")` annotation directly above your class definition.
- **Deployment Descriptor (Legacy):** Configuring mappings inside the `web.xml` file using `<servlet>` and `<servlet-mapping>` tags.

APPLICATION STRUCTURE

```
LoginExample/
 ├─ src/com/login/
 │    ├─ LoginServlet.java      (servlet, mapped to /login)
 │    ├─ DBconection.java       (JDBC connection)
 │    └─ PasswordUtil.java      (PBKDF2 password hashing)
 └─ webapp/
      ├─ login.html             (form, posts to "login")
      ├─ welcome.html           (success page)
      ├─ images/welcome.jpg     (image used by welcome.html)
      └─ WEB-INF/
           ├─ web.xml
           └─ lib/mysql-connector-j-<version>.jar
```

DATABASE SETUP

Run `schema.sql` once in the MySQL client. The two sample users are the ones visible in the supplied database screenshot (admin / karan); the passwords are `adm` and `kk`, and only their salted PBKDF2 hashes are stored. These are lab-only passwords.

schema.sql SQL

```sql
-- Database and table expected by the login application
CREATE DATABASE IF NOT EXISTS loginDB;
USE loginDB;

CREATE TABLE IF NOT EXISTS users (
    id       INT          PRIMARY KEY,
    username VARCHAR(50)  NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL    -- PBKDF2 hash: iterations:saltHex:hashHex
);

-- Sample users for the lab (passwords: admin -> adm, karan -> kk).
-- Only the hashes are stored; generate your own with
--   java -cp classes com.login.PasswordUtil <password>
INSERT INTO users (id, username, password) VALUES
    (1,   'admin',
'120000:90808c96ec43f18e080020ac27ef0e6c:a5e2d8c4a455966b124c13a33467dcd6bcfd4ae3e29d35
c18245b63797a7ce25'),
    (101, 'karan',
'120000:4a152f6ef5516966f94c3e09fabb5a43:2c348e7d39a85c4b1b2a93bfa9efdaa7edd4e3d8d423e0
fd908947ccfe20ccd4');
```

CODE

Servlet Class

com/login/LoginServlet.java JAVA

```java
package com.login;

import java.io.IOException;
import java.io.PrintWriter;
import java.security.GeneralSecurityException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
```

```
import java.sql.SQLException;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        response.setContentType("text/html");
        response.setCharacterEncoding("UTF-8");

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        // empty or missing values are rejected up front.
        if (username == null || username.isEmpty()
                || password == null || password.isEmpty()) {
            showFailure(response);
            return;
        }

        // the stored PBKDF2 hash of the user is fetched with
        // a PreparedStatement and the entered password is verified against
        // it. Both username and password must be correct.
        String sql = "SELECT password FROM users WHERE username = ?";

        try (Connection con = DBconection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, username);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()
                        && PasswordUtil.verify(password, rs.getString("password"))) {
                    response.sendRedirect("welcome.html");
                } else {
                    showFailure(response);
                }
            }

        } catch (SQLException | GeneralSecurityException | RuntimeException e) {
            // details go to the server log only,
            // never to the browser.
            getServletContext().log("Login processing error", e);
            response.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
            PrintWriter out = response.getWriter();
            out.println("<html>");
            out.println("<body>");
            out.println("<h1>Database Error</h1>");
            out.println("<p>Unable to process the request. "
```

```
                    + "Please try again later.</p>");
            out.println("</body>");
            out.println("</html>");
        }
    }

    private void showFailure(HttpServletResponse response) throws IOException {
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head>");
        out.println("<title>Login Failed</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h1>Login Failed</h1>");
        out.println("<p>Invalid username or password.</p>");
        out.println("<a href='login.html'>Try Again</a>");
        out.println("</body>");
        out.println("</html>");
    }
}
```

Database Connection Class

com/login/DBconection.java JAVA

```
package com.login;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBconection {

    // values come from environment variables; the
    // placeholders must be replaced by your own MySQL user and password.
    private static final String URL =
            System.getenv().getOrDefault("DB_URL",
                    "jdbc:mysql://localhost:3306/loginDB?useSSL=false");
    private static final String USER =
            System.getenv().getOrDefault("DB_USER", "your_mysql_user");
    private static final String PASSWORD =
            System.getenv().getOrDefault("DB_PASSWORD", "your_mysql_password");

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

Password Hashing Class

com/login/PasswordUtil.java JAVA

```
package com.login;

import java.security.GeneralSecurityException;
import java.security.MessageDigest;
import java.security.SecureRandom;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;

/**
```

```
 * Password hashing with PBKDF2-HMAC-SHA256 (standard JDK classes only).
 * Stored format:  iterations:saltHex:hashHex
 */
public class PasswordUtil {

    private static final String ALGORITHM = "PBKDF2WithHmacSHA256";
    private static final int ITERATIONS = 120000;
    private static final int SALT_BYTES = 16;
    private static final int KEY_BITS = 256;

    public static String hash(String password) throws GeneralSecurityException {
        byte[] salt = new byte[SALT_BYTES];
        new SecureRandom().nextBytes(salt);
        byte[] key = derive(password, salt, ITERATIONS);
        return ITERATIONS + ":" + toHex(salt) + ":" + toHex(key);
    }

    public static boolean verify(String password, String stored)
            throws GeneralSecurityException {
        if (password == null || stored == null) {
            return false;
        }
        String[] parts = stored.split(":");
        if (parts.length != 3) {
            return false;
        }
        int iterations = Integer.parseInt(parts[0]);
        byte[] salt = fromHex(parts[1]);
        byte[] expected = fromHex(parts[2]);
        byte[] actual = derive(password, salt, iterations);
        return MessageDigest.isEqual(expected, actual);   // constant-time
    }

    private static byte[] derive(String password, byte[] salt, int iterations)
            throws GeneralSecurityException {
        PBEKeySpec spec = new PBEKeySpec(password.toCharArray(), salt,
                iterations, KEY_BITS);
        return SecretKeyFactory.getInstance(ALGORITHM)
                .generateSecret(spec).getEncoded();
    }

    private static String toHex(byte[] data) {
        StringBuilder sb = new StringBuilder();
        for (byte b : data) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }

    private static byte[] fromHex(String hex) {
        byte[] out = new byte[hex.length() / 2];
        for (int i = 0; i < out.length; i++) {
            out[i] = (byte) Integer.parseInt(hex.substring(2 * i, 2 * i + 2), 16);
        }
        return out;
    }

    // Helper to create hashes for the sample rows:
    //   java -cp . com.login.PasswordUtil <password>
```

```
    public static void main(String[] args) throws Exception {
        System.out.println(hash(args[0]));
    }
}
```

Login.html

login.html HTML

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Login</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f2f2f2;
        }
        .login-container {
            width: 350px;
            margin: 100px auto;
            padding: 25px;
            background-color: white;
            border-radius: 8px;
            box-shadow: 0 0 10px #ccc;
        }
        h2 {
            text-align: center;
        }
        input[type="text"],
        input[type="password"] {
            width: 100%;
            padding: 10px;
            margin: 8px 0 15px 0;
            box-sizing: border-box;
        }
        input[type="submit"] {
            width: 100%;
            padding: 10px;
            background-color: #007bff;
            color: white;
            border: none;
            cursor: pointer;
            border-radius: 4px;
        }
        input[type="submit"]:hover {
            background-color: #0056b3;
        }
    </style>
</head>
<body>
<div class="login-container">
    <h2>Login</h2>
    <form action="login" method="post">
        <label>Username:</label>
        <input type="text"
               name="username"
               required>
        <label>Password:</label>
        <input type="password"
```

```
                name="password"
                required>
        <input type="submit" value="login">
    </form>
</div>
</body>
</html>
```

Welcome.html

welcome.html HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Welcome Page</title>
</head>
<body>
    <h1>Welcome to My Website!</h1>
    <p>Login Successful!</p>
    <p>You have successfully logged into the system.</p>
    <img src="images/welcome.jpg"
         width="400"
         height="250"
         alt="Welcome Image">
    <br><br>
    <h2>Have a Great Day!</h2>
</body>
</html>
```

Web.xml

WEB-INF/web.xml XML

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
         https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
         version="6.0">
    <welcome-file-list>
        <welcome-file>login.html</welcome-file>
    </welcome-file-list>
</web-app>
```

DEPLOYMENT AND RUN INSTRUCTIONS

1. Install JDK 11 or above and Apache Tomcat 10.1.x. Servlet classes that import `jakarta.servlet` do not run on Tomcat 9 or older.
2. Execute `schema.sql` in MySQL.
3. Copy the MySQL Connector/J JAR into `WEB-INF/lib`.
4. Set the environment variables `DB_USER` and `DB_PASSWORD` for the Tomcat process (or replace the placeholders in `DBconection.java` on your own machine; never submit real passwords).
5. Deploy the project (Run on Server in the IDE, or a WAR file in Tomcat's `webapps` folder) and open `http://localhost:8080/LoginExample/`.

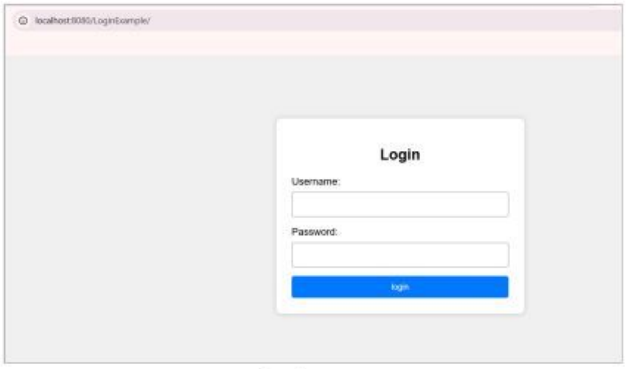
6. Place an image named `welcome.jpg` in `images/`; it is referenced by `welcome.html` but is not part of the supplied files.

TEST CASES

#	Username	Password	Expected	Result
1	admin	adm	Redirect to welcome.html	HTTP 302 → welcome.html
2	karan	kk	Redirect to welcome.html	HTTP 302 → welcome.html
3	admin	wrong	Login Failed page	"Invalid username or password."
4	admin	ADM	Login Failed (case-sensitive)	"Invalid username or password."
5	nouser	adm	Login Failed	"Invalid username or password."
6	(empty)	(empty)	Login Failed	"Invalid username or password."
7	`' OR '1'='1`	`' OR '1'='1`	Login Failed (SQL injection rejected)	"Invalid username or password."
8	`admin' --`	anything	Login Failed (SQL injection rejected)	"Invalid username or password."
9	admin	adm	Generic error page when the table is unavailable; no SQL details in the browser	HTTP 500 "Database Error"; no exception text in the page; details only in the server log
10	—	—	`GET /login` is not supported (servlet has only `doPost`)	HTTP 405

OUTPUT

Username: **admin** Password: **adm**. In the database, the username and password are checked. If they match, the welcome page is displayed.



Login page



Welcome page after a successful login

```
1 • show databases;
2 • use logindb;
3 • show tables;
4 • select * from users;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	username	password
101	karan	kk
1	admin	adm

Database: *users table with sample records.*



Login failed: *page shown for a wrong password.*

CONCLUSION

VIVA QUESTIONS

1. What is a servlet and who manages its life cycle?
2. Name the life-cycle methods of a servlet and say how many times each is called.
3. What is the difference between `doGet()` and `doPost()`? Why is POST used for a login form?
4. How is a servlet mapped to a URL (annotation and `web.xml`)?
5. Why is a `PreparedStatement` used instead of building the SQL text by concatenation?
6. Why should passwords be stored as salted hashes instead of plain text?

EXPERIMENT NO. 5

Student Information Display using JSP

AIM

To develop a JSP page that accepts student details through an HTML form and displays the entered student information using **JSP implicit objects**.

PROBLEM STATEMENT

Develop a JSP page to accept and display student details such as **Roll Number, Name, Branch, Year, Email, and Mobile Number** using the JSP implicit request object.

SOFTWARE REQUIREMENTS

- JDK 8 or above (JDK 11 or above with Tomcat 10.1)
- Apache Tomcat Server
- Eclipse / NetBeans / IntelliJ IDEA
- JSP
- HTML
- Web Browser

THEORY

Life Cycle of JSP

Stages of the JSP Life Cycle

- **Translation:** The web container takes the `.jsp` file and converts it into a Java servlet source code file (`.java`).
- **Compilation:** The container compiles the generated `.java` servlet file into a bytecode class file (`.class`).
- **Class Loading:** The servlet container's class loader loads the compiled `.class` file into memory.
- **Instantiation:** The container creates an object (instance) of the loaded servlet class.
- **Initialization (`jspInit()`):** The container calls the `jspInit()` method once to set up resources like database connections or configuration parameters.
- **Request Processing (`_jspService()`):** The container invokes the `_jspService()` method for every client request, passing the HttpServletRequest and HttpServletResponse objects to generate dynamic content.
- **Destruction (`jspDestroy()`):** The container calls the `jspDestroy()` method right before removing the JSP from service to clean up open files or close database connections.

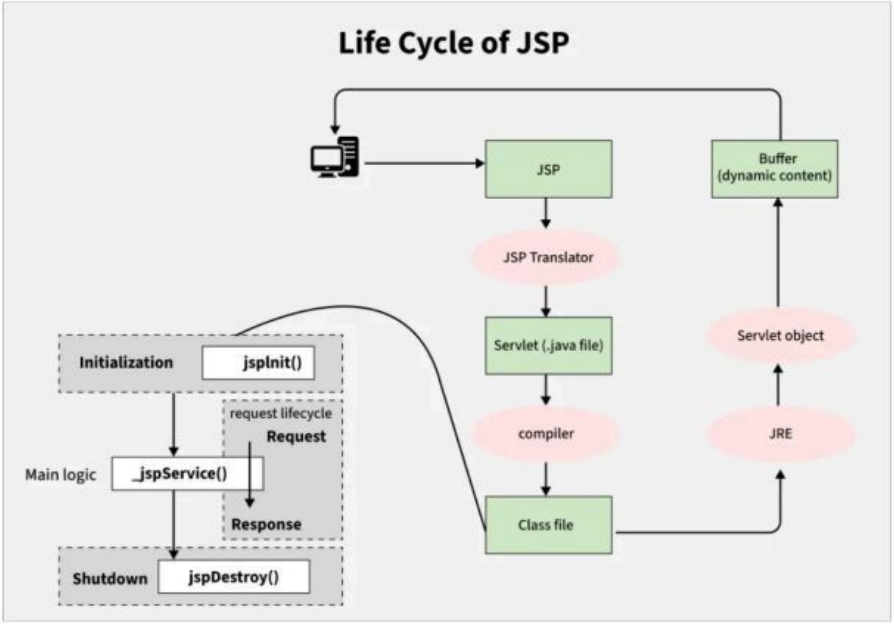


Fig: *Life Cycle of JSP*

JSP Implicit Objects – request and response

Implicit objects are pre-defined Java objects created by the JSP container and made available to every JSP page automatically. These objects can be used directly without declaration, simplifying development.

- Both are implicit objects (no need to declare)
- Created automatically by JSP container
- Used for client-server communication

The implicit objects are:

- **request**: instance of HttpServletRequest
- **response**: instance of HttpServletResponse
- **config**: instance of ServletConfig
- **application**: instance of ServletContext
- **session**: instance of HttpSession
- **pageContext**: instance of PageContext
- **page**: reference to the current JSP page object
- **exception**: instance of Throwable (used in error pages)
- **out**: instance of JspWriter

Request Object

The request implicit object is an instance of HttpServletRequest. It is created by the web container for each client request and is used to:

- Read form data
- Access request parameters
- Retrieve request headers
- Obtain session information

Response Object

The **response implicit object** is an instance of **HttpServletResponse**. It is created by the web container for each client request and is used to:

- Send response data back to the client
- Set the response content type
- Write HTML or other output to the browser
- Set response headers
- Send error messages or redirect the client

Advantage of JSP over Servlet

- JSP is easier to write and maintain compared to servlets
- Separation of presentation (HTML) and logic (Java)
- Implicit objects reduce boilerplate code
- Faster development for UI-centric applications

CODE

Source file: `StudentInformation.jsp`. The form fields and the JSP read the same parameter names: `rollno`, `name`, `branch`, `year`, `email`, `mobile`.

StudentInformation.jsp JSP

```jsp
<%@ page language="java"
         contentType="text/html; charset=UTF-8" %>
<%!
    // HTML-escape every value before it is written to the page
    // (prevents entered text from being interpreted as HTML / script).
    private static String esc(String s) {
        if (s == null) return "";
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray()) {
            switch (c) {
                case '&':  sb.append("&amp;");  break;
                case '<':  sb.append("&lt;");   break;
                case '>':  sb.append("&gt;");   break;
                case '"':  sb.append("&quot;"); break;
                case '\'': sb.append("&#39;");  break;
                default:   sb.append(c);
            }
        }
        return sb.toString();
    }

    private static String val(String s) {
        return (s == null) ? "" : s.trim();
    }

    private static String sel(String current, String option) {
        return option.equals(current) ? " selected" : "";
    }
%>
<%
    request.setCharacterEncoding("UTF-8");
```

```
    // Check whether form is submitted
    boolean submitted = request.getParameter("rollno") != null;

    String rollno = val(request.getParameter("rollno"));
    String name   = val(request.getParameter("name"));
    String branch = val(request.getParameter("branch"));
    String year   = val(request.getParameter("year"));
    String email  = val(request.getParameter("email"));
    String mobile = val(request.getParameter("mobile"));

    // server-side validation (the original had none).
    java.util.List<String> errors = new java.util.ArrayList<String>();
    if (submitted) {
        if (!rollno.matches("[0-9]{1,10}"))
            errors.add("Roll Number must contain 1 to 10 digits.");
        if (!name.matches("[A-Za-z][A-Za-z .'-]{1,49}"))
            errors.add("Student Name must contain letters only (2 to 50 characters).");
        if (branch.isEmpty())
            errors.add("Branch is required.");
        if (year.isEmpty())
            errors.add("Year is required.");
        if (!email.matches("[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}"))
            errors.add("Enter a valid Email address.");
        if (!mobile.matches("[6-9][0-9]{9}"))
            errors.add("Mobile Number must be a 10-digit number starting with 6-9.");
    }
%>
<html>
<head>
    <title>Student Information</title>
</head>
<body>

    <h2>Student Information Form</h2>

    <form method="post">

        Roll Number:
        <input type="text" name="rollno" value="<%= esc(rollno) %>">
        <br><br>

        Student Name:
        <input type="text" name="name" value="<%= esc(name) %>">
        <br><br>

        Branch:
        <select name="branch">
            <option value="Electronics Engineering"<%= sel(branch, "Electronics
Engineering") %>>Electronics Engineering</option>
            <option value="Computer Engineering"<%= sel(branch, "Computer Engineering")
%>>Computer Engineering</option>
            <option value="Electronics and Computer Engineering"<%= sel(branch,
"Electronics and Computer Engineering") %>>Electronics and Computer
Engineering</option>
        </select>
        <br><br>

        Year:
        <select name="year">
            <option value="First Year"<%= sel(year, "First Year") %>>First
Year</option>
```

```
            <option value="Second Year"<%= sel(year, "Second Year") %>>Second
Year</option>
            <option value="Third Year"<%= sel(year, "Third Year") %>>Third
Year</option>
            <option value="Final Year"<%= sel(year, "Final Year") %>>Final
Year</option>
        </select>
        <br><br>

        Email:
        <input type="text" name="email" value="<%= esc(email) %>">
        <br><br>

        Mobile Number:
        <input type="text" name="mobile" value="<%= esc(mobile) %>">
        <br><br>

        <input type="submit" value="Display Information">

    </form>

<%
    if (submitted && !errors.isEmpty()) {
%>
        <hr>
        <h3>Please correct the following:</h3>
        <ul>
<%      for (String err : errors) { %>
            <li><%= esc(err) %></li>
<%      } %>
        </ul>
<%
    } else if (submitted) {
%>
        <hr>
        <h2>Student Information</h2>

        <p><b>Roll Number:</b> <%= esc(rollno) %></p>
        <p><b>Student Name:</b> <%= esc(name) %></p>
        <p><b>Branch:</b> <%= esc(branch) %></p>
        <p><b>Year:</b> <%= esc(year) %></p>
        <p><b>Email:</b> <%= esc(email) %></p>
        <p><b>Mobile Number:</b> <%= esc(mobile) %></p>

        <hr>
        <h3>Student information displayed successfully.</h3>
<%
    }
%>

</body>
</html>
```

DEPLOYMENT AND RUN INSTRUCTIONS

1. Create a Dynamic Web Project (for example `JSP_Tags`) and place `StudentInformation.jsp` in the web content folder.
2. Run it on Apache Tomcat and open `http://localhost:8080/JSP_Tags/StudentInformation.jsp`.

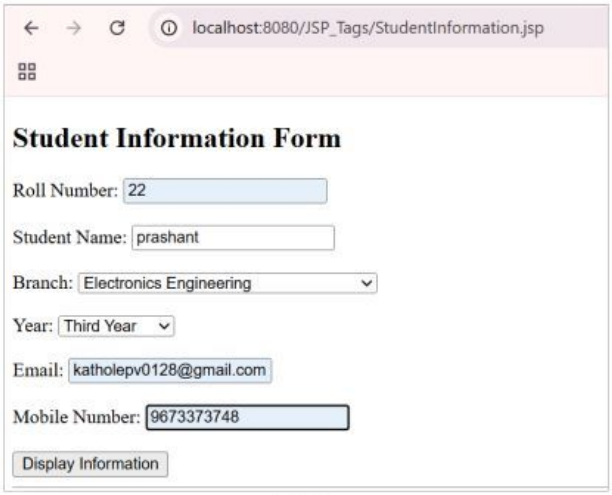
3. No database or extra library is needed.

TEST CASES

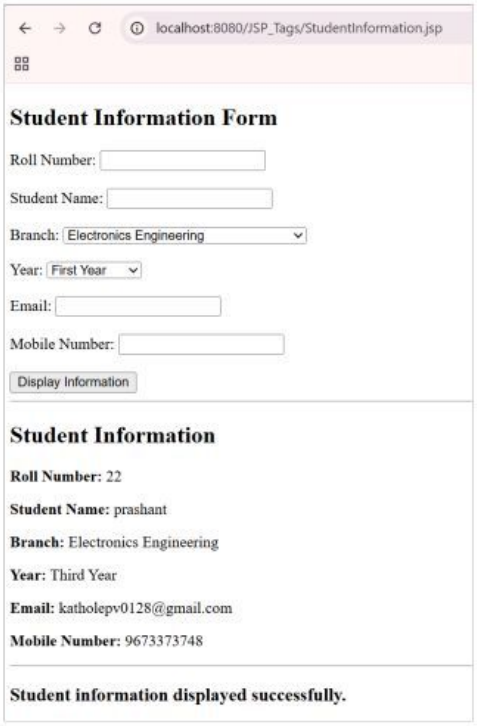
#	Input	Expected	Result
1	Roll 22, name prashant, Electronics Engineering, Third Year, a valid e-mail, mobile 9673373748	All six values displayed	Student Information block with the six values and the success line
2	All fields empty	Error list, no information block	Six messages (roll number, name, branch, year, e-mail, mobile)
3	Roll 22A, name `<b>Asha</b>`, e-mail `asha@`, mobile 12345	Errors for roll number, name, e-mail and mobile	Four messages; the typed name is shown as text, not as bold HTML
4	Name `<script>alert(1)</script>` with other fields valid	No script reaches the page	Rejected by validation; the field is re-filled as `<script>…`; no executable `<script>` tag in the response
5	E-mail containing `"><img onerror=…>`	Not executable	No `<img` tag in the response
6	Page opened with GET (no form data)	Only the form is shown	Form only

OUTPUT

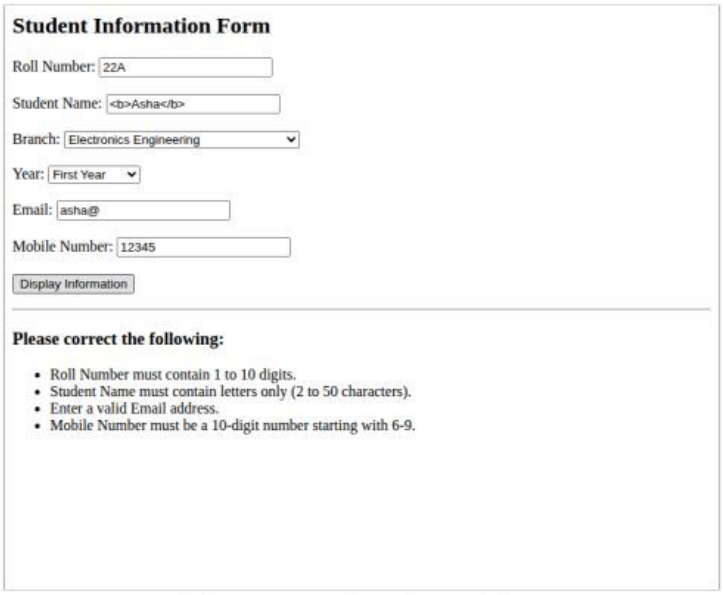
After pressing the *Display Information* button, the following output is obtained.



Filled form



Displayed information



Validation errors *shown for invalid input.*

CONCLUSION

VIVA QUESTIONS

1. Name the stages of the JSP life cycle.
2. What are implicit objects? Name any five.
3. How does `request.getParameter()` work and what does it return when the parameter is absent?
4. What is the difference between the scriptlet tag `<% %>` and the expression tag `<%= %>`?
5. What is the advantage of JSP over a servlet?
6. What is cross-site scripting (XSS) and how does HTML escaping prevent it?

EXPERIMENT NO. 6

JSP-Based Calculator for Arithmetic Operations

AIM

To design and develop a **JSP-based web application** that performs basic arithmetic operations such as **addition, subtraction, multiplication, and division** on two numbers entered by the user.

PROBLEM STATEMENT

Create a JSP-based calculator that accepts two numbers from the user and performs the selected arithmetic operation. The application should display the calculated result on the JSP page.

SOFTWARE REQUIREMENTS

- JDK 8 or above (JDK 11 or above with Tomcat 10.1)
- Apache Tomcat Server
- Eclipse IDE / NetBeans / IntelliJ IDEA
- JSP
- HTML
- Web Browser

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Internet connection is not mandatory

THEORY

Scriptlet Tag

The Scriptlet Tag in JSP is used to embed Java code directly into a JSP page. The code written inside the `<% %>` tag is executed on the server whenever a request is made, helping generate dynamic content. It is one of the basic scripting elements provided by JSP.

- Allows writing control statements like loops and conditions (if, for, while)
- Can access request parameters and session data
- Used for processing logic, not just displaying output

Expression Tag

The Expression Tag in JSP is used to display output directly on the client's browser. It allows embedding Java expressions inside a JSP page and automatically converts them into string output. This tag simplifies printing data without writing explicit output statements.

- Uses `<%= %>` syntax to write Java expressions
- Automatically prints output to the browser (no need for `out.print()`)

- Mainly used for displaying data, not for writing logic

Declaration Tag

The Declaration Tag in JSP is used to declare variables, methods, and classes in a JSP page. The code written inside this tag is placed outside the `_jspService()` method by the JSP container, making it available throughout the JSP page.

- Uses `<%! %>` syntax to declare variables, methods, and classes
- Declarations become class-level members of the generated servlet
- Used for defining reusable logic, not for direct output

CODE

Source file: `Calculator.jsp`. The form fields `num1` and `num2` are the parameters read by the scriptlet.

Calculator.jsp JSP

```jsp
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<head>
    <title>Simple JSP Calculator</title>
</head>
<body>

    <h2>Simple JSP Calculator</h2>

    <form method="post">

        Enter First Number:
        <input type="text" name="num1">
        <br><br>

        Enter Second Number:
        <input type="text" name="num2">
        <br><br>

        <input type="submit" value="Calculate">

    </form>

<%
    // Check whether the form is submitted
    if (request.getParameter("num1") != null &&
        request.getParameter("num2") != null) {

        // input is validated; a missing, non-numeric,
        // NaN or Infinity value no longer causes a server error.
        double n1 = 0, n2 = 0;
        boolean valid = true;
        try {
            n1 = Double.parseDouble(request.getParameter("num1").trim());
            n2 = Double.parseDouble(request.getParameter("num2").trim());
            if (Double.isNaN(n1) || Double.isInfinite(n1) ||
                Double.isNaN(n2) || Double.isInfinite(n2)) {
                valid = false;
            }
        } catch (NumberFormatException e) {
            valid = false;
```

```
        }

        if (!valid) {
%>
        <p style="color:red;">Please enter two valid numbers.</p>
<%
        } else {
            double addition = n1 + n2;
            double subtraction = n1 - n2;
            double multiplication = n1 * n2;
%>
        <h3>Calculation Results</h3>

        <p>Addition = <%= addition %></p>
        <p>Subtraction = <%= subtraction %></p>
        <p>Multiplication = <%= multiplication %></p>
<%
            if (n2 != 0) {
%>
        <p>Division = <%= n1 / n2 %></p>
<%
            } else {
%>
        <p>Division = Cannot divide by zero</p>
<%
            }
        }
    }
%>

</body>
</html>
```

DEPLOYMENT AND RUN INSTRUCTIONS

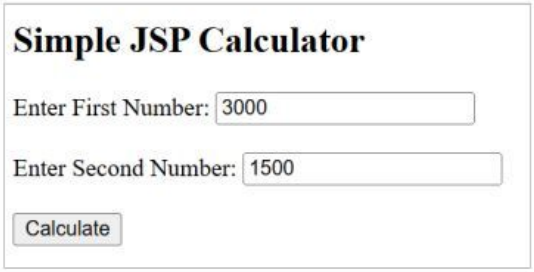
1. Place `Calculator.jsp` in the web content folder of the project (for example `JSP_Tags`).
2. Run it on Apache Tomcat and open `http://localhost:8080/JSP_Tags/Calculator.jsp`.
3. No database or extra library is needed.

TEST CASES

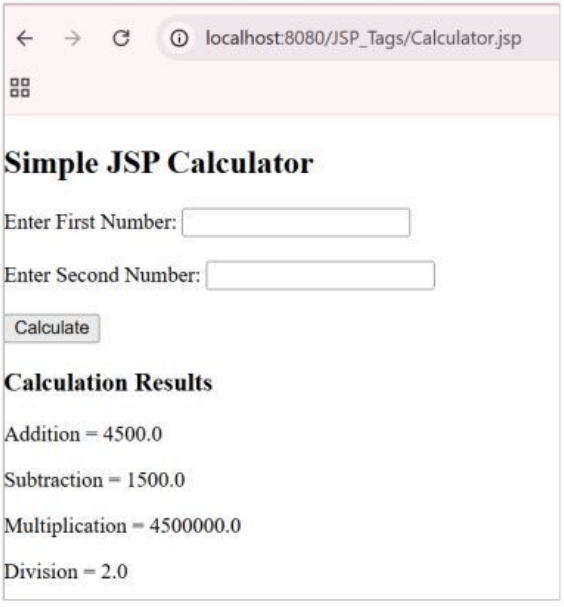
#	First number	Second number	Expected	Result
1	3000	1500	4500, 1500, 4500000, 2.0	Addition = 4500.0; Subtraction = 1500.0; Multiplication = 4500000.0; Division = 2.0
2	10	0	Division reported as impossible	Division = Cannot divide by zero (other three results shown)
3	-7.5	2.5	-5, -10, -18.75, -3	Addition = -5.0; Subtraction = -10.0; Multiplication = -18.75; Division = -3.0
4	0	5	Division 0	Division = 0.0
5	1e3	4	Scientific notation accepted	1004.0; 996.0; 4000.0; 250.0
6	abc	5	Error message, no server error	“Please enter two valid numbers.”
7	5	(empty)	Error message	“Please enter two valid numbers.”
8	NaN / Infinity	2 / 1	Error message	“Please enter two valid numbers.”

OUTPUT

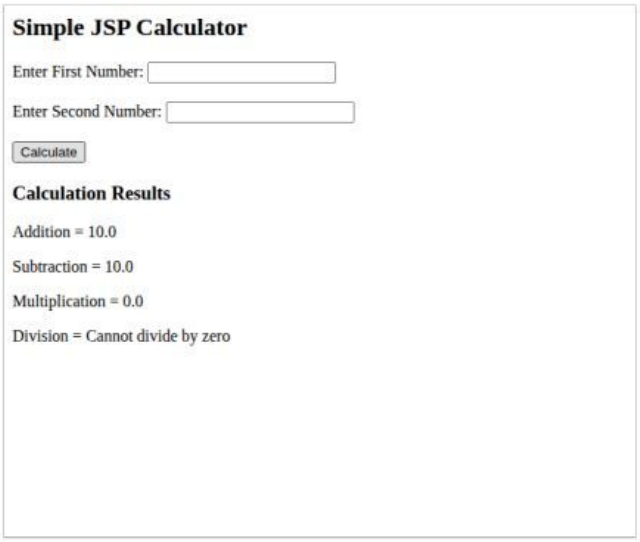
After pressing the *Calculate* button, the following output is displayed.



Inputs 3000 and 1500



Calculation results



Division by zero: *inputs 10 and 0.*

CONCLUSION

VIVA QUESTIONS

1. What are the JSP scripting elements?
2. What is the difference between the scriptlet, expression and declaration tags?
3. Where does the code of a declaration tag go in the generated servlet?
4. How are the two numbers passed from the form to the JSP page?
5. Why is `Double.parseDouble()` wrapped in a `try` block here?
6. How does the program avoid dividing by zero?